

6.5610 CHEAT SHEET

Lecture 1: One-Way Hash Functions

Deterministic, public, fixed length output
 $h: \{0,1\}^* \rightarrow \{0,1\}^d$ Ideal: random oracle
In practice: PRFs.

Security Properties:

- One-way** (preimage resistance)
Given y , infeasible to find any x s.t. $h(x)=y$
Needed for: password storage (store $h(pw)$)
- Collision Resistance**
Infeasible to find $x \neq x'$ s.t. $h(x)=h(x')$
Needed for: digital signatures, commitments
sign $h(m)$ (2nd preimage resistance)
Given x , infeasible to find $x' \neq x$ s.t. $h(x')=h(x)$
Needed for: file integrity (compare $h(f)$)
- Target Collision Resistance**
Given x_1 , infeasible to find $x_2 \neq x_1$ s.t. $h(x_2)=h(x_1)$
Needed for: file integrity (compare $h(f)$)

Relations between Properties:

- $CR \Rightarrow TCR$ ✓ but not vice versa
- $CR \not\Rightarrow OW$ $h(x)=f(x) \parallel x$ is CR but reveals x
- $OW \not\Rightarrow CR$ $H(x_1, x_2) = f(x_1)$ is OW but collides

General Attacks (space of 2^n values)

- Birthday attack: finds collisions
time $\approx 2^{n/2}$. Sample $\sim 2^{n/2}$ outputs;
expect collision by birthday paradox.
- Inversion: breaks OW
time $O(2^n)$ by brute-force
preimage search.

Exercise Hints:

- OWF: $g(x_1, x_2) = f(x_1) \otimes x_2$
Given $y = f(x) \otimes x_2$, to get $g(x_1, x_2') = 0 \dots 0$
 g is not one-way
- CR: $h(x, y) = h(x, h_2(y))$
If either h_1 or h_2 collides, h collides.
Since both h_1 and h_2 are CR, h is CR.

Lecture 4: PRPs and AES

Pseudorandom Permutation (PRP):

$P: \mathcal{K} \times \mathcal{X} \rightarrow \mathcal{X}$
Correctness: $P(k, P(k, x)) = x \forall k, x$
 $PRPADv[A, P] := |Pr[W_0 = 1] - Pr[W_1 = 1]|$
 $b = 0: P(k) \leftarrow P(k, x)$
 $b = 1: P(k) \leftarrow P(k, x) \oplus m$

PRP Switching Lemma: (Using PRP as a PRF)

$PRPADv[A, P] \leq PRPADv[A, P] + T/2^n$
If P is a secure PRP and $T \ll 2^n$, then P is also a secure PRF. Use PRP in CTR mode like PRF.
Limit: Once $T \approx 2^{n/2}$, lemma becomes vacuous.
Birthday paradox: adversary queries $2^{n/2}$ points and checks for collisions.
PRF has collisions, PRP doesn't.
For AES ($n=128$): stay well below 2^{64} invocations per key.

Even-Mansour Cipher:

Key: $(K_0, K_1) \in \{0,1\}^n \times \{0,1\}^n$
(2n-bit Key)
Block: n bits
 $P_{EM}(K_0, K_1, x) = K_1 \oplus \Pi(x \oplus K_0)$

AES as iterated Even-Mansour

Derive round keys $K_0, \dots, K_r$
each linear in input key K
 $st \leftarrow x \oplus K_0$
for $i = 1, \dots, r$:
 $st \leftarrow \Pi(st) \oplus K_i$
output st
128-bit key $\Rightarrow r=10$ rounds

AES Round Permutation Π :

- SubBytes: Apply fixed S-box $S: \{0,1\}^8 \rightarrow \{0,1\}^8$ to each of the 16 bytes. (non-linear)
Need AES-NI hardware
- Shift Rows: View 16 bytes as 4×4 matrix. Cyclically shift row by i positions right ($i = 0, 1, 2, 3$).
- MixColumns: Multiply 4×4 byte matrix by a fixed matrix over $GF(2^8)$.

Lecture 6: Rabin, RSA, why post-quantum crypto?

Rabin Cryptosystem (1979):

KeyGen: p, q prime (typically choose $p, q \approx 2^{n/2}$)
 $n = pq, \phi(n) = (p-1)(q-1)$
 $Enc(n, m) = m^2 \pmod{n}$
 $Dec(SK, c) = \{r \in \mathbb{Z}^n : r^2 \equiv c \pmod{n}\}$
 $S = \mathbb{Z}_n^{\times/2} \pmod{n}$
Use CRT combinators:
 $m \equiv \pm 1 \pmod{p}$
 $m \equiv \pm 1 \pmod{q}$
4 plaintexts $m \in \{m_1, m_2, m_3, m_4\}$

Security Equivalence

to factoring: If the adversary finds two roots x, y with $x^2 \equiv y^2 \pmod{n}$ but $x \not\equiv \pm y \pmod{n}$ then $gcd(x-y, n)$ yields p, q .
Breaking Rabin = factoring n .
Weaknesses:
- 4 roots: receiver must use padding/context for correct m .
- Deterministic: not IND-CPA without randomized padding.
- CPA broken: oracle returns different roots, attacker factors n .

Lecture 7: LWE

LWE Assumption (Post-quantum hard):

params: n, m, q (prime), error dist χ
 $A \in \mathbb{Z}_q^{n \times m}, s \in \mathbb{Z}_q^m$
 $u \in \mathbb{Z}_q^m$ (uniform)
LWE: $(A, As + e) \approx (A, u)$ want even if $m \gg n$, to
When LWE is trivially broken: not find s easily.
 χ : uniform over $GF(q)$: e is large, useless
 χ : always 0: no noise, solves s directly.

Attacks on LWE:

- Lattice Reduction: Convert LWE to shortest vector problem (SVP). Best known: B^2 algo. $2^{0.02n}$
- BKW Algorithm: Divide and conquer, eliminates secret coordinates by summing LWE sample pairs $2O(\frac{n}{\log n})$ requires exponential samples.
- Quantum Algorithms: No known poly-time quantum algo. for LWE.

Lecture 2: Symmetric Encryption

$Enc_K: \mathcal{K} \times \mathcal{M} \rightarrow \mathcal{C}$
 $Dec_K: \mathcal{K} \times \mathcal{C} \rightarrow \mathcal{M}$
Correctness: $Pr[Dec_K(K, Enc_K(K, m)) = m] = 1$
(Enc may be randomized)
Security: $\approx$ (indistinguishable to PPT)
Enc must be randomized
Computation indistinguishability:
families $A = \{A_n\}, B = \{B_n\}, A \approx B$ if for every PPT distinguisher D :
 $|Pr[D(A_n) = 1] - Pr[D(B_n) = 1]| \leq \text{negl}(n)$
 D wins guess which distribution game with prob $\leq 1/2 + \text{negl}(n)$.

CPA Security (Chosen Plaintext Attack):

- Challenger chooses $K \leftarrow \mathcal{K}$
 - A queries messages m_i adaptively, receives $c_i \leftarrow Enc_K(K, m_i)$. Poly many times
 - A chooses challenge pair m_0, m_1 .
 - Challenger picks $b \in \{0,1\}$ sends $c \leftarrow Enc_K(K, m_b)$
 - A outputs b' . Wins if $b' = b$.
- Secure: $\Pr[A \text{ wins}] \leq 1/2 + \text{negl}(n)$

Pseudorandom function (PRF):

$|Pr[A^{F(K)}(x) = 1] - Pr[A^{R(x)}(x) = 1]| \leq \text{negl}(n)$

PRF $\Rightarrow$ CPA-Secure Encryption

$Enc_K(m) = (r, m \oplus F(K, r))$ $r \leftarrow \mathcal{X}_\lambda$

$Dec_K(r, c) = F(K, r) \oplus c$

PRF $\Rightarrow$ MAC (integrity)

Append tag $T = F(K, c)$ to ciphertext c .

Receiver verifies tag before accepting.

PRF security ensures adversary can't forge valid tag T without key K .

MAC $\Rightarrow$ verify msg integrity, prevent tampering, unlike encryption only protects confidentiality.

MAC $\Rightarrow$ verify msg integrity, prevent tampering, unlike encryption only protects confidentiality.

MAC $\Rightarrow$ verify msg integrity, prevent tampering, unlike encryption only protects confidentiality.

MAC $\Rightarrow$ verify msg integrity, prevent tampering, unlike encryption only protects confidentiality.

MAC $\Rightarrow$ verify msg integrity, prevent tampering, unlike encryption only protects confidentiality.

MAC $\Rightarrow$ verify msg integrity, prevent tampering, unlike encryption only protects confidentiality.

MAC $\Rightarrow$ verify msg integrity, prevent tampering, unlike encryption only protects confidentiality.

MAC $\Rightarrow$ verify msg integrity, prevent tampering, unlike encryption only protects confidentiality.

MAC $\Rightarrow$ verify msg integrity, prevent tampering, unlike encryption only protects confidentiality.

MAC $\Rightarrow$ verify msg integrity, prevent tampering, unlike encryption only protects confidentiality.

MAC $\Rightarrow$ verify msg integrity, prevent tampering, unlike encryption only protects confidentiality.

MAC $\Rightarrow$ verify msg integrity, prevent tampering, unlike encryption only protects confidentiality.

MAC $\Rightarrow$ verify msg integrity, prevent tampering, unlike encryption only protects confidentiality.

MAC $\Rightarrow$ verify msg integrity, prevent tampering, unlike encryption only protects confidentiality.

MAC $\Rightarrow$ verify msg integrity, prevent tampering, unlike encryption only protects confidentiality.

MAC $\Rightarrow$ verify msg integrity, prevent tampering, unlike encryption only protects confidentiality.

MAC $\Rightarrow$ verify msg integrity, prevent tampering, unlike encryption only protects confidentiality.

MAC $\Rightarrow$ verify msg integrity, prevent tampering, unlike encryption only protects confidentiality.

MAC $\Rightarrow$ verify msg integrity, prevent tampering, unlike encryption only protects confidentiality.

MAC $\Rightarrow$ verify msg integrity, prevent tampering, unlike encryption only protects confidentiality.

MAC $\Rightarrow$ verify msg integrity, prevent tampering, unlike encryption only protects confidentiality.

MAC $\Rightarrow$ verify msg integrity, prevent tampering, unlike encryption only protects confidentiality.

MAC $\Rightarrow$ verify msg integrity, prevent tampering, unlike encryption only protects confidentiality.

MAC $\Rightarrow$ verify msg integrity, prevent tampering, unlike encryption only protects confidentiality.

MAC $\Rightarrow$ verify msg integrity, prevent tampering, unlike encryption only protects confidentiality.

MAC $\Rightarrow$ verify msg integrity, prevent tampering, unlike encryption only protects confidentiality.

MAC $\Rightarrow$ verify msg integrity, prevent tampering, unlike encryption only protects confidentiality.

MAC $\Rightarrow$ verify msg integrity, prevent tampering, unlike encryption only protects confidentiality.

MAC $\Rightarrow$ verify msg integrity, prevent tampering, unlike encryption only protects confidentiality.

MAC $\Rightarrow$ verify msg integrity, prevent tampering, unlike encryption only protects confidentiality.

MAC $\Rightarrow$ verify msg integrity, prevent tampering, unlike encryption only protects confidentiality.

MAC $\Rightarrow$ verify msg integrity, prevent tampering, unlike encryption only protects confidentiality.

MAC $\Rightarrow$ verify msg integrity, prevent tampering, unlike encryption only protects confidentiality.

MAC $\Rightarrow$ verify msg integrity, prevent tampering, unlike encryption only protects confidentiality.

MAC $\Rightarrow$ verify msg integrity, prevent tampering, unlike encryption only protects confidentiality.

MAC $\Rightarrow$ verify msg integrity, prevent tampering, unlike encryption only protects confidentiality.

MAC $\Rightarrow$ verify msg integrity, prevent tampering, unlike encryption only protects confidentiality.

MAC $\Rightarrow$ verify msg integrity, prevent tampering, unlike encryption only protects confidentiality.

MAC $\Rightarrow$ verify msg integrity, prevent tampering, unlike encryption only protects confidentiality.

MAC $\Rightarrow$ verify msg integrity, prevent tampering, unlike encryption only protects confidentiality.

MAC $\Rightarrow$ verify msg integrity, prevent tampering, unlike encryption only protects confidentiality.

MAC $\Rightarrow$ verify msg integrity, prevent tampering, unlike encryption only protects confidentiality.

MAC $\Rightarrow$ verify msg integrity, prevent tampering, unlike encryption only protects confidentiality.

MAC $\Rightarrow$ verify msg integrity, prevent tampering, unlike encryption only protects confidentiality.

MAC $\Rightarrow$ verify msg integrity, prevent tampering, unlike encryption only protects confidentiality.

MAC $\Rightarrow$ verify msg integrity, prevent tampering, unlike encryption only protects confidentiality.

MAC $\Rightarrow$ verify msg integrity, prevent tampering, unlike encryption only protects confidentiality.

MAC $\Rightarrow$ verify msg integrity, prevent tampering, unlike encryption only protects confidentiality.

MAC $\Rightarrow$ verify msg integrity, prevent tampering, unlike encryption only protects confidentiality.

MAC $\Rightarrow$ verify msg integrity, prevent tampering, unlike encryption only protects confidentiality.

MAC $\Rightarrow$ verify msg integrity, prevent tampering, unlike encryption only protects confidentiality.

MAC $\Rightarrow$ verify msg integrity, prevent tampering, unlike encryption only protects confidentiality.

MAC $\Rightarrow$ verify msg integrity, prevent tampering, unlike encryption only protects confidentiality.

MAC $\Rightarrow$ verify msg integrity, prevent tampering, unlike encryption only protects confidentiality.

MAC $\Rightarrow$ verify msg integrity, prevent tampering, unlike encryption only protects confidentiality.

MAC $\Rightarrow$ verify msg integrity, prevent tampering, unlike encryption only protects confidentiality.

MAC $\Rightarrow$ verify msg integrity, prevent tampering, unlike encryption only protects confidentiality.

MAC $\Rightarrow$ verify msg integrity, prevent tampering, unlike encryption only protects confidentiality.

MAC $\Rightarrow$ verify msg integrity, prevent tampering, unlike encryption only protects confidentiality.

MAC $\Rightarrow$ verify msg integrity, prevent tampering, unlike encryption only protects confidentiality.

MAC $\Rightarrow$ verify msg integrity, prevent tampering, unlike encryption only protects confidentiality.

Lecture 5: Diffie-Hellman and ElGamal

Diffie-Hellman Key Exchange:

Public cyclic group G , generator g . Establishes shared secret against passive eavesdropper Eve.

1 Alice picks $x \in \{0, \dots, |G|-1\}$, sends g^x .

2 Bob picks $y \in \{0, \dots, |G|-1\}$, sends g^y .

3 Alice: $K = (g^y)^x = g^{xy}$ Bob: $K = (g^x)^y = g^{xy}$

Eve sees only g^x, g^y . Key derivation: use K to derive two subkeys: one for enc, one for MAC.

Computational Diffie-Hellman (CDH) Assumption: given g, g^x, g^y , hard to compute g^{xy} . Eve can't compute K , but may learn partial info (e.g. MSB).

Decisional Diffie-Hellman (DDH) Assumption: given g, g^x, g^y , hard to distinguish g^{xy} from g^u (u random).

From DDH, K is indistinguishable from random $\Rightarrow$ safe to use for enc/MAC.

MITM Attack: Eve shares g^x to Alice & Bob and reads all the traffic, need authenticated public keys.

IND-CCA2:

1 A picks m_0, m_1 . Challenger returns $C^* = Enc(pk, m_b)$

2 A may query decryption oracle on any $ct \neq C^*$ (poly times even after seeing C^*)

3 A outputs b' . Wins if $b' = b$.
Secure: $\Pr[b' = b] \leq 1/2 + \text{negl}$.

Secure $\Rightarrow$ $\Pr[b' = b] \leq 1/2 + \text{negl}$.

Secure $\Rightarrow$ $\Pr[b' = b] \leq 1/2 + \text{negl}$.

Secure $\Rightarrow$ $\Pr[b' = b] \leq 1/2 + \text{negl}$.

Secure $\Rightarrow$ $\Pr[b' = b] \leq 1/2 + \text{negl}$.

Secure $\Rightarrow$ $\Pr[b' = b] \leq 1/2 + \text{negl}$.

Secure $\Rightarrow$ $\Pr[b' = b] \leq 1/2 + \text{negl}$.

Secure $\Rightarrow$ $\Pr[b' = b] \leq 1/2 + \text{negl}$.

Secure $\Rightarrow$ $\Pr[b' = b] \leq 1/2 + \text{negl}$.

Secure $\Rightarrow$ $\Pr[b' = b] \leq 1/2 + \text{negl}$.

Secure $\Rightarrow$ $\Pr[b' = b] \leq 1/2 + \text{negl}$.

Secure $\Rightarrow$ $\Pr[b' = b] \leq 1/2 + \text{negl}$.

Secure $\Rightarrow$ $\Pr[b' = b] \leq 1/2 + \text{negl}$.

Secure $\Rightarrow$ $\Pr[b' = b] \leq 1/2 + \text{negl}$.

Secure $\Rightarrow$ $\Pr[b' = b] \leq 1/2 + \text{negl}$.

Secure $\Rightarrow$ $\Pr[b' = b] \leq 1/2 + \text{negl}$.

Secure $\Rightarrow$ $\Pr[b' = b] \leq 1/2 + \text{negl}$.

Secure $\Rightarrow$ $\Pr[b' = b] \leq 1/2 + \text{negl}$.

Secure $\Rightarrow$ $\Pr[b' = b] \leq 1/2 + \text{negl}$.

Secure $\Rightarrow$ $\Pr[b' = b] \leq 1/2 + \text{negl}$.

Secure $\Rightarrow$ $\Pr[b' = b] \leq 1/2 + \text{negl}$.

Secure $\Rightarrow$ $\Pr[b' = b] \leq 1/2 + \text{negl}$.

Secure $\Rightarrow$ $\Pr[b' = b] \leq 1/2 + \text{negl}$.

Secure $\Rightarrow$ $\Pr[b' = b] \leq 1/2 + \text{negl}$.

Secure $\Rightarrow$ $\Pr[b' = b] \leq 1/2 + \text{negl}$.

Secure $\Rightarrow$ $\Pr[b' = b] \leq 1/2 + \text{negl}$.

Secure $\Rightarrow$ $\Pr[b' = b] \leq 1/2 + \text{negl}$.

Secure $\Rightarrow$ $\Pr[b' = b] \leq 1/2 + \text{negl}$.

Secure $\Rightarrow$ $\Pr[b' = b] \leq 1/2 + \text{negl}$.

Secure $\Rightarrow$ $\Pr[b' = b] \leq 1/2 + \text{negl}$.

Secure $\Rightarrow$ $\Pr[b' = b] \leq 1/2 + \text{negl}$.

Secure $\Rightarrow$ $\Pr[b' = b] \leq 1/2 + \text{negl}$.

Secure $\Rightarrow$ $\Pr[b' = b] \leq 1/2 + \text{negl}$.

Secure $\Rightarrow$ $\Pr[b' = b] \leq 1/2 + \text{negl}$.

Secure $\Rightarrow$ $\Pr[b' = b] \leq 1/2 + \text{negl}$.

Secure $\Rightarrow$ $\Pr[b' = b] \leq 1/2 + \text{negl}$.

Secure $\Rightarrow$ $\Pr[b' = b] \leq 1/2 + \text{negl}$.

Secure $\Rightarrow$ $\Pr[b' = b] \leq 1/2 + \text{negl}$.

Secure $\Rightarrow$ $\Pr[b' = b] \leq 1/2 + \text{negl}$.

Secure $\Rightarrow$ $\Pr[b' = b] \leq 1/2 + \text{negl}$.

Secure $\Rightarrow$ $\Pr[b' = b] \leq 1/2 + \text{negl}$.

Secure $\Rightarrow$ $\Pr[b' = b] \leq 1/2 + \text{negl}$.

Secure $\Rightarrow$ $\Pr[b' = b] \leq 1/2 + \text{negl}$.

Secure $\Rightarrow$ $\Pr[b' = b] \leq 1/2 + \text{negl}$.

Secure $\Rightarrow$ $\Pr[b' = b] \leq 1/2 + \text{negl}$.

Secure $\Rightarrow$ $\Pr[b' = b] \leq 1/2 + \text{negl}$.

Secure $\Rightarrow$ $\Pr[b' = b] \leq 1/2 + \text{negl}$.

Secure $\Rightarrow$ $\Pr[b' = b] \leq 1/2 + \text{negl}$.

Secure $\Rightarrow$ $\Pr[b' = b] \leq 1/2 + \text{negl}$.

Secure $\Rightarrow$ $\Pr[b' = b] \leq 1/2 + \text{negl}$.

Secure $\Rightarrow$ $\Pr[b' = b] \leq 1/2 + \text{negl}$.

Secure $\Rightarrow$ $\Pr[b' = b] \leq 1/2 + \text{negl}$.

Secure $\Rightarrow$ $\Pr[b' = b] \leq 1/2 + \text{negl}$.

Secure $\Rightarrow$ $\Pr[b' = b] \leq 1/2 + \text{negl}$.

Secure $\Rightarrow$ $\Pr[b' = b] \leq 1/2 + \text{negl}$.

Secure $\Rightarrow$ $\Pr[b' = b] \leq 1/2 + \text{negl}$.

Secure $\Rightarrow$ $\Pr[b' = b] \leq 1/2 + \text{negl}$.

Secure $\Rightarrow$ $\Pr[b' = b] \leq 1/2 + \text{negl}$.

Secure $\Rightarrow$ $\Pr[b' = b] \leq 1/2 + \text{negl}$.

Secure $\Rightarrow$ $\Pr[b' = b] \leq 1/2 + \text{negl$

Lecture 9 & 10: Fully Homomorphic Encryption

FHE Definition: sk: decryption key (secret)
KeyGen(1^n) $\rightarrow$ (zk, ek) ek: evaluation key (public)
Enc(s, m in $\mathbb{Z}_2$) $\rightarrow$ ct F: boolean circuit in class C
Dec(s, ct) $\rightarrow$ m $\hat{c}t$: encrypts $F(m_1, \dots, m_\ell)$
Eval($ek, F, ct_1, \dots, ct_\ell$) $\rightarrow$ $\hat{c}t$
Message space = bits. Computation model = Boolean circuits (XOR + AND). XOR/AND are mod-2 complete.
Three Required Properties: With high probability $\rightarrow$ w.h.p.
Correctness: Dec($sk, Eval(ek, F, ct)$) = $F(m_1, \dots, m_\ell)$ w.h.p.
no correctness $\rightarrow$ non-homomorphic scheme trivially works.
Semantic Security: $(ek, Enc(sk, 0)) \approx (ek, Enc(sk, 1))$ even **no security $\rightarrow$ send plaintext**
Compactness: $|\hat{c}t|$ depends only on n , not on $|F|$ or ℓ .
no compactness $\rightarrow$ Eval just outputs $(F, ct_1, \dots, ct_\ell)$ and Dec does work.

Homomorphic Encryption Landscapes:
• Additive only: XOR gates only. LWE-based scheme from L8 (linear hom) $Enc(m_0) + Enc(m_1) = Enc(m_0 + m_1)$
• Multiplicative only: AND gates only. ElGamal, RSA are multiplicatively homomorphic: $Enc(m_0) \cdot Enc(m_1) = Enc(m_0 * m_1)$
• Fully Homomorphic (FHE): Both XOR & AND $\rightarrow$ arbitrary bool circuits

GSW Scheme:
Ideal (Insecure) version:
SK = vector S. Ciphertext = matrix C where S is exact eigenvector.
C = $\lambda \cdot S$ λ : message
Add: $(C_1 + C_2)S = (\lambda_1 + \lambda_2)S$ Multiply: $(C_1 \cdot C_2)S = (\lambda_1 \cdot \lambda_2)S$
Broken: efficient eigenvector algorithms exist.
Approximate (LWE-secure) version:
Replace exact eigenvector with noisy version:
C = $\mu Gs + e$ (decryption invariant)
 μ : message bit e : small error G : public gadget matrix
Must preserve this invariant across ADD $\times$ MUL gates with bounded error growth.

GSW Construction:
 $t = (n+1) \log(q)$
KeyGen:
 $S \in \mathbb{Z}_q^{n \times t}$
 $SK = (S^{-1}, 1)^T \in \mathbb{Z}_q^{(n+1) \times t}$
 $Enc(s, m \in \{0,1\})$:
 $A \in \mathbb{Z}_q^{n \times t}$
 $e \leftarrow \chi^t$
 $B = [AS + e] \in \mathbb{Z}_q^{(n+1) \times t}$
 $C = B \cdot M$ output C
Dec(S, C):
 $V = C \cdot S$
if all entries of V small ($< q/4$): output 0
else: output 1.

Homomorphic Evaluation:
ADD Gate: Eval(ct_1, ct_2) = $C_1 + C_2$
 $(C_1 + C_2)S = C_1S + C_2S$
 $= (\mu_1 Gs + e_1) + (\mu_2 Gs + e_2)$
 $= (\mu_1 + \mu_2)Gs + (e_1 + e_2)$
new message: $\mu_1 + \mu_2$
new error: $e_1 + e_2$
error growth $\geq 2x$ per addition.
MUL Gate: Eval(X, C_1, C_2) = $h(C_1) \cdot C_2$
 $h(C_1) \cdot C_2 \cdot S = h(C_1) \cdot (\mu_2 Gs + e_2)$
 $= \mu_2 \cdot h(C_1) \cdot Gs + h(C_1) \cdot e_2$
 $= (\mu_1 \mu_2)Gs + (\mu_1 e_2 + h(C_1) e_2)$
new message: $\mu_1 \mu_2$
 $h(C_1)$ has binary entries
 $\rightarrow$ error growth $\approx (n+1) \log(q)$ per mul.

Levelled FHE-Depth Limit & Error Budget:
Error growth rate per gate type:
• Add: error doubles ($\times 2$)
• Mul: error grows by $\times (n+1) \log(q) \approx n \log(q)$
After d multiplications from initial error B :
 $|e| \leq (n+1) \log(q)^d \cdot B$
Decryption succeeds while $|e| < q/4$.
Bootstrapping: noisy but detectable $C \rightarrow$ fresh low-noise C' without encrypting μ (still encrypting μ) learning μ .
① Model Dec as a boolean circuit C_{pec} with inputs $s \in \mathbb{Z}_q^{n+1}$ and C .
② Server already knows $C \rightarrow$ hard code to get C_{pec} . C only takes S as input.
③ User publishes $ek = (Enc(s, S_1), \dots, Enc(s, S_n)) \rightarrow$ secret key bits encrypted under themselves.
④ Server homomorphically evaluates C_{pec} .
on ek to get $C' \leftarrow Eval(C_{pec}, C, ek)$
noise reset to $(t^4 \cdot B_0)$ (regardless of circuit depth)
Why C' has low noise
noise in C' depends only on (1) freshness of ek entries and (2) depth of C_{pec}
Both are fixed and small - independent of #gates preceded by bootstrap.
GSW Bootstrapping feasibility
 C_{pec} depth = $O(\log n)$ (comparison + log-depth adder tree). depth $n^{poly} \approx O(\log n)$
Requirement: levelled FHE must support depth $\geq d_{pec} + 1$ (decrypt circuit's plus one gate)

Circular Security:
Def: Enc is circular secure if an adversary cannot break semantic security even given $ek = \{Enc(s, s_i)\} \rightarrow$ the secret key bits encrypted under themselves.

Open Problem: GSW circular security cannot be proven from LWE alone. FHE without this assumption is still an open problem.

IND-CPA doesn't imply circular security.
Counter example: Construct T' from any IND-CPA scheme T :
 $Enc'(m) = (SK, 1)$ if $m = SK$. $(Enc(m), 0)$ otherwise
IND-CPA holds: adversary picks m_0, m_1 before seeing SK , so $Pr(m_0 = SK) = 0$ challenge = $(Enc(m_0), 0)$
Circular security fails: $Enc'(SK) = (SK, 1)$ directly reveals SK .
ElGamal also not assumed circular secure: $SK \times \log(q)$ encrypting algebraic functions of x can leak structure via group operations.

FHE Applications:
• Private delegation: outsource computation to untrusted server without revealing input. ex. private LLM queries
• Secure collaboration: Multiple parties jointly evaluate a function, revealing only the output. ex. private ML training across hospitals. Sealed-bid auctions.
• Private DB Lookup: Generalization of PIR. Arbitrary queries (SQL, web search) answered by server under encryption.

Open Questions in FHE:
① FHE from non-lattice assumptions? $\rightarrow$ Only known from LWE/RingLWE diversity of assumptions = resilience
② Concretely efficient FHE? $\rightarrow$ Low multiplicative degree (≤ 3) + high additive degree is feasible today. High multi-degree requires large params + expensive bootstrapping.
FHE not yet practical at scale. Boolean circuit representation and $O(poly(n))$ per-gate overhead are the bottleneck.

Lecture 11: Zero Knowledge & Interactive Proofs

Interactive Proof (IP):
Prover P (possibly inefficient) + Verifier V (ppt)
Exchange messages $m_1, \dots, m_k$ sequentially.
V has private randomness r not revealed to P.
Completeness: $\forall x \in L, Pr(P(V, x) = 1) \geq 2/3$
Soundness: $\forall x \notin L, \forall P, Pr(P(V, x) = 1) \leq 1/3$
 $2/3 \approx 1/3$ are arbitrary. Repeat λ times, majority vote gives completeness $1 - \text{negl}(\lambda)$ and soundness $\text{negl}(\lambda)$
Key distinctions:
• Remove randomness $\rightarrow$ deterministic $\rightarrow$ class collapses to NP
• Remove interaction (keep randomness) $\rightarrow$ class AM
• Both present $\rightarrow$ PSPACE (Shamir '90) (believed = NP)

Zero Knowledge:
 $\forall$ (malicious) $V^*, \exists$ ppt. Sim st. $\forall x \in L$:
 $\{View_{V^*}(P(x), V^*)\} \approx \{Sim(x)\}$
View = $\#$ of randomness + all messages received from P.
ZK: verifier learns nothing beyond truth of the statement $\rightarrow$ could've generated transcript alone.
3-Coloring ZK proof:
① Prover picks random permutation π of $\{1, 2, 3\}$.
Commits color $\pi(i)$ for each vertex in a locked box.
② Verifier challenges with random edge $(i, j) \in E$.
③ Prover reveals key to boxes i & j only.
④ Verifier accepts iff colors are distinct + valid.
Completeness = 1 Soundness = $1 - 1/|E|$ per round
Amplify by repetition, fresh π each time.
ZK: verifier sees only 2 distinct random colors, can simulate without prover.

Commitment Schemes:
Gen(1^n) $\rightarrow$ pp Com(pp, m, r) $\rightarrow$ C. Open with (m, r)
Properties:
• Hiding: $(pp, Com(m_0, r_0)) \approx (pp, Com(m_1, r_1))$
• Binding: no ppt. adversary can find m_0, m_1 s.t. $Com(m_0, r_0) = Com(m_1, r_1)$ except with negl. prob.
LWE-based bit commitment: computationally hiding
Gen: $A \in \mathbb{Z}_q^{m \times n}, u \in \mathbb{Z}_q^n$ statistically binding
PP: (A, u) Hiding: LWE $\rightarrow (A, AS+e) \approx (A, u)$
Com(pp, b, s, e):
 $s \in \mathbb{Z}_q^m, e \in \mathbb{Z}_q^n$ adding fixed u preserves pseudo randomness regardless of b .
 $C = As + e + bu \in \mathbb{Z}_q^m$ Binding: Double opening require $u \pm (A \cdot \Delta s + \Delta e)$. The set of such u has size $q^{O(n)} \cdot 2^{O(t)}$. So random u hits it with negl. prob.

Lecture 12: Secret Sharing
T-out-of-N Secret Sharing:
Share(m) $\rightarrow (S_1, \dots, S_n)$ Randomized
Reconstruct($\{(S_i, S_i)_{i \in I}\}$) $\rightarrow m$ $|I| = t$
Correctness: any t shares reconstruct m exactly.
Security: any $< t$ shares are identically distributed for all msgs m, m' .

n-out-of-n scheme (XOR):
Choose $s_1, \dots, s_{n-1} \in \mathbb{R}^t$ uniformly;
Set $S_n = m \oplus s_1 \oplus \dots \oplus s_{n-1}$
Reconstruct: XOR all shares. Any $n-1$ shares are uniform and independent of m .

Shamir's Secret Sharing:
Share(m) over $G[F_p]$, $p > n$:
Choose $a_1, \dots, a_{t-1} \in \mathbb{R}^t$ $G[F_p]$
 $f(x) = m + a_1 x + \dots + a_{t-1} x^{t-1}$
 $S_i = f(i)$ for $i \in [n]$
Reconstruct (Lagrange interpolation):
 $f(x) = \sum_i S_i \cdot f_i(x)$ $m = f(0)$
 $f_i(x) = \prod_{j \neq i} \frac{0 - x_j}{0 - x_i}$
Example: $h=3, t=2, m=10$ over $G[F_{11}]$
 $f(x) = 2x + 10 \pmod{11}$
 $S_1 = (1, 1)$ $S_2 = (2, 3)$ $S_3 = (3, 5)$
Reconstruct from $\{S_2, S_3\}$:
 $f(0) = 3(3) + 5(-2) = 10 \pmod{11}$
 $x=0: f_2(0) = \frac{3-0}{2-0} = \frac{3}{2} = 3$

Connection to Reed-Solomon codes:
Shamir SS is essentially a Reed-Solomon code. RS encodes message $(m_0, \dots, m_{t-1})$ as polynomial evaluations on all p field points. Can decode even if up to $\frac{p-t+1}{2}$ coordinates are maliciously corrupted (not just erased). In MPC malicious setting: ask all n parties for shares, tolerate $< \frac{n-t+1}{2}$ bad shares.

Lecture 13: Multiparty Computation MPC (BGW)

MPC:
Secure iff $\forall$ ppt adversary A (real world), $\exists$ ppt simulator S (ideal world) s.t.: $Real_A(x_1, \dots, x_n) \equiv Ideal_S(x_1, \dots, x_n)$
Ideal world: parties send inputs to trusted party, who computes and returns $f(x_1, \dots, x_n)$. Real $\equiv$ ideal means real protocol leaks nothing beyond the output.
Two Adversary Models:
• Semi-Honest $\rightarrow$ Follows protocol honestly but tries to learn from transcripts. BGW secure against $< n/2$ corruptions.
• Malicious $\rightarrow$ can send arbitrary msgs. BGW secure against $< n/3$ corruptions.
BGW is information-theoretically secure. No cryptographic assumptions. Secure even against computationally unbounded adversaries. (with crypto: can tolerate any number of corruptions)
BGW Protocol (Ben-Or, Goldwasser, Wigderson)
Phase 1: input secret sharing
Each party P_i creates degree $t-1$ polynomial $q_i(x)$ with $q_i(0) = S_i$. Sends $q_i(j)$ to party P_j . Adversary controlling $\leq t-1$ parties learn nothing (Shamir Security)
Phase 2: gate-by-gate computation
Add: P_i sends new share $= p(i) + q(i)$. Local, no interaction. Degree unchanged.
MUL: Requires interaction. Degree doubles from $t-1$ to $2t-2$; must reduce back.

BGW Multiplication:
① Local Multiply: $h(i) = p(i) \cdot q(i)$ points on degree $2t-2$ polynomial $h(x) = p(x)q(x)$, $h(0) = ab$.
② Reshare: Each P_i picks random degree $t-1$ poly $g_i(x)$ with $g_i(0) = h(i)$ sends $g_i(j)$ to P_j .
③ Linear Recombination: P_j computes $S_j = \sum_i \lambda_i \cdot U_{ij} \pmod{p}$ where λ_i are Lagrange coefficients for $x=0$ of h . $\lambda_i = \frac{1}{h(0) - h(i)}$
Result: degree $t-1$ sharing of ab .
Requires $n \geq 2t-1$. With $t_{adv} = t-1$ this gives $t_{adv} < n/2$.
Privacy Bound Summary:
Shamir: $|A| \leq t-1 \rightarrow$ zero info about secret
Shamir: $|B| \geq t \rightarrow$ full reconstruction
BGW semi-honest: $d \geq t_{adv}$ and $n \geq 2d+1 \rightarrow t_{adv} < n/2$

Lecture 14: MPC in Practice

MPC in Practice:
• **Crypto Custody** (MPC wallets): Signing key split via secret sharing across employees. Transactions require threshold signing MPC. Special-purpose "threshold signing" protocols used - much faster than BGW. Vulnerable to software bugs (all parties run same code) $\rightarrow$ one exploitable bug compromises all parties at once.
• **Private JOIN** (Google, Meta): Ad conversion measurement $\rightarrow$ Google holds ad-exposed users, BAW holds purchasers. MPC computes intersection size without either party revealing its list. Uses ad-hoc protocol, not BGW.
• **Private Aggregation** (Apple, Mozilla): Millions of clients each hold x_i . Compute $\sum x_i$ (or histogram) without learning individual values. Addition-only circuit (no MUL). Largest scale MPC deployments in practice.

Client-Server Model:
Why BGW breaks at scale: Comm cost scales with # MUL gates $\times$ circuit depth. Millions of parties $\rightarrow$ infeasible pairwise communication.
Solution: $K \ll n$ servers $\rightarrow$ Each client secret shares input to K servers ($K \geq 2$ typical). Servers run MPC amongst themselves. Per-client comm cost = $O(K)$, independent of n . Downside: K Servers colluding breaks all privacy.
Private Aggregation: (Boyle-Gilboa-Ishai)
Setting: Each client submits a one-hot vector $x_i \in \{0,1\}^m$ as additive shares X_i, X_j to 2 servers. Servers sum shares to get histogram.
Problem: malicious clients can submit non-one-hot vectors.
Sketch Verification Protocol:
1. Servers agree on random $r, R \in \mathbb{F}^m$ ($R_i = r \cdot x_i$) unknown to client.
2. Each server computes inner products of its share: $tA = \langle X, r \rangle$, $TA = \langle X, R \rangle$
3. Check: $(tA + tB) - (TA + TB) = 0$ (and likewise for B)
Only $O(1)$ field elements communicated.
4. Accept and include in histogram, or reject & discard.
Properties:
• Completeness $\rightarrow$ valid one-hot x satisfies algebraic relation exactly. Always accepted
• Soundness $\rightarrow$ invalid x fails with prob $1 - O(1/|F|)$ by Schwartz-Zippel: a nonzero degree- d poly over $\mathbb{F}$ has $\leq d/|F|$ chance of vanishing on random input.
• Privacy (HVZK) $\rightarrow$ Servers learn only accept/reject. Transcript simulatable given outcome.

Complexity:
Client comm: $O(1)$ field elements
Server compute: $O(m)$ field ops
Independence: $O(1)$ is independent of m .

Security is information theoretic:
 $< t$ shares reveal zero bits about the secret, regardless of adversary's computational power.
Threshold intuition: with $t-1$ shares the distribution over possible secrets is uniform (perfectly flat). Adding the t -th share collapses it to a single point.

Why Reconstruction is Unique:
The coefficient matrix of t equations in t unknowns is a Vandermonde matrix with det = $\prod_{i < j} (x_j - x_i) \neq 0$ (distinct evaluation points).
Invertible $\rightarrow$ unique polynomial $\rightarrow$ unique secret

Public Coins: V sends only uniformly-sampled field elements - no private randomness needed. Enables Fiat-Shamir transform to remove interaction entirely (non-interactive proofs).

Sumcheck Soundness:
Soundness error $\leq \frac{m-d}{|F|}$ for large field (eg. 128-bit primes) + moderate m , soundness error negl.

Formultilinear $f(d=1) \leq m/|F|$
Key lemma: Schwartz-Zippel
A nonzero univariate poly of degree $\leq d$ has at most d roots in $\mathbb{F}$. So if $g_1 \neq g_2$ (cheating prover differs from truth at root t_0), $Pr(g_1(t_0) = g_2(t_0)) \leq d/|F|$.
Union bound over m rounds gives $m \cdot d/|F|$.
Cheating Example: $(F_2, f(x_1, x_2) = x_1 x_2)$
True sum $S = 4$. Round 1: honest $G(x) = 1+2x$.
 V sends $r_1 = 5$, $g_1(5) = 11 \neq 4$.
Round 2: cheating P sends $g_2(x) = 4x$ instead of $5+x$.
Sumcheck: $g_1(0) + g_2(1) = 0 + 4 = 4 = g_1(5)$.
 V sends $r_2 = 3$.
Final check: $g_2(3) = 12 \neq 5$, but $f(5, 3) = 8 \neq 1$ $5 \neq 1$ $\times$
Intermediate round checks can be fooled, but final oracle query catches the lie.
Doubly Efficient Interactive Proofs: (DE-IP)
Def: For $L \in DTIME(T(n))$:
Prover Runtime = $poly(T)$ (ideally $O(T)$)
Verifier Runtime = $polylog(T) + \tilde{O}(n)$.
Both prover & verifier efficient. (unlike classical IP where prover is all-powerful.)
GKR Protocol (Goldwasser-Kalai-Rothblum):
Built entirely from sumcheck. For log-space uniform circuit of depth D , size S :
Rounds: $D \cdot polylog(S)$
Comm: $D \cdot polylog(S)$
Verifier: $O(n) + D \cdot polylog(S)$
Prover: $poly(S)$

Guest Lectures: Jim Bidzos + Ron Rivest on RSA.

Lecture 15: SumCheck

Sumcheck Problem:
Given: $f: \mathbb{F}^m \rightarrow \mathbb{F}$, degree $\leq d$ per variable
Fixed set $H \subseteq \mathbb{F}$ (often $H = \{0,1\}$)
Claim: $\beta = \sum_{h_1, \dots, h_m \in H} f(h_1, \dots, h_m)$
Goal: V has oracle access to f ; verify claim in time $O(|H|^m)$.
Sumcheck is a key building block: used in PSPACE (Shamir), GKR protocol, and many succinct proof systems.
Why it's powerful: $|H|^m$ can be exponentially large. $H = \{0,1\}$, $m = 128 \rightarrow 2^{128}$ terms. Verifier checks the sum in $O(m \cdot d)$ time instead of $O(|H|^m)$. Trades sequential interaction for exponential speedup.

Sumcheck Protocol - Round Structure: m rounds, one per variable.
① P sends $g_1(x) = \sum_{h_2, \dots, h_m \in H} f(x, h_2, \dots, h_m)$ univariate, deg $\leq d$.
② V checks $deg(g_1) \leq d$ & $\sum_{h_2, \dots, h_m \in H} g_1(h_2, \dots, h_m)$.
③ P sends $g_2(x) = \sum_{h_3, \dots, h_m \in H} f(x_2, x_3, \dots, h_m)$.
④ V checks $deg(g_2) \leq d$ & $\sum_{h_3, \dots, h_m \in H} g_2(h_3, \dots, h_m)$.
Repeat for variables $x_3, \dots, x_m$.
⑤ P sends $g_m(x) = f(x_1, \dots, x_m)$. V samples t_m checks $g_m(t_m) = f(t_m)$ via oracle. Accepts iff all checks pass.

Complexity:
Rounds: m
Communication: $O(d \log(|F|))$
Verifier time: $O(m \cdot |H| \cdot d \cdot polylog(|F|))$
Public Coins: V sends only uniformly-sampled field elements - no private randomness needed. Enables Fiat-Shamir transform to remove interaction entirely (non-interactive proofs).

Sumcheck Soundness:
Soundness error $\leq \frac{m-d}{|F|}$ for large field (eg. 128-bit primes) + moderate m , soundness error negl.

Formultilinear $f(d=1) \leq m/|F|$
Key lemma: Schwartz-Zippel
A nonzero univariate poly of degree $\leq d$ has at most d roots in $\mathbb{F}$. So if $g_1 \neq g_2$ (cheating prover differs from truth at root t_0), $Pr(g_1(t_0) = g_2(t_0)) \leq d/|F|$.
Union bound over m rounds gives $m \cdot d/|F|$.
Cheating Example: $(F_2, f(x_1, x_2) = x_1 x_2)$
True sum $S = 4$. Round 1: honest $G(x) = 1+2x$.
 V sends $r_1 = 5$, $g_1(5) = 11 \neq 4$.
Round 2: cheating P sends $g_2(x) = 4x$ instead of $5+x$.
Sumcheck: $g_1(0) + g_2(1) = 0 + 4 = 4 = g_1(5)$.
 V sends $r_2 = 3$.
Final check: $g_2(3) = 12 \neq 5$, but $f(5, 3) = 8 \neq 1$ $5 \neq 1$ $\times$
Intermediate round checks can be fooled, but final oracle query catches the lie.
Doubly Efficient Interactive Proofs: (DE-IP)
Def: For $L \in DTIME(T(n))$:
Prover Runtime = $poly(T)$ (ideally $O(T)$)
Verifier Runtime = $polylog(T) + \tilde{O}(n)$.
Both prover & verifier efficient. (unlike classical IP where prover is all-powerful.)
GKR Protocol (Goldwasser-Kalai-Rothblum):
Built entirely from sumcheck. For log-space uniform circuit of depth D , size S :
Rounds: $D \cdot polylog(S)$
Comm: $D \cdot polylog(S)$
Verifier: $O(n) + D \cdot polylog(S)$
Prover: $poly(S)$